



Data vs Model Parallelism

Scaling Training Across GPUs and Nodes

A technical deep dive into parallel training strategies for large-scale machine learning models that exceed single-GPU capabilities.

Why Revisit Parallelism Strategies Now?



Early DL Era

Single-GPU training was sufficient for most deep learning workloads



LLM Revolution

Foundation models with billions of parameters exceed single-GPU memory capacity



Mixed Approaches

Efficient scaling now requires sophisticated combinations of **data**, **model**, and **pipeline parallelism**

Your choice of parallelism strategy directly impacts **throughput**, **memory utilization**, and ultimately **training costs**.

Data Parallelism (DP)

The simplest and most widely adopted approach:

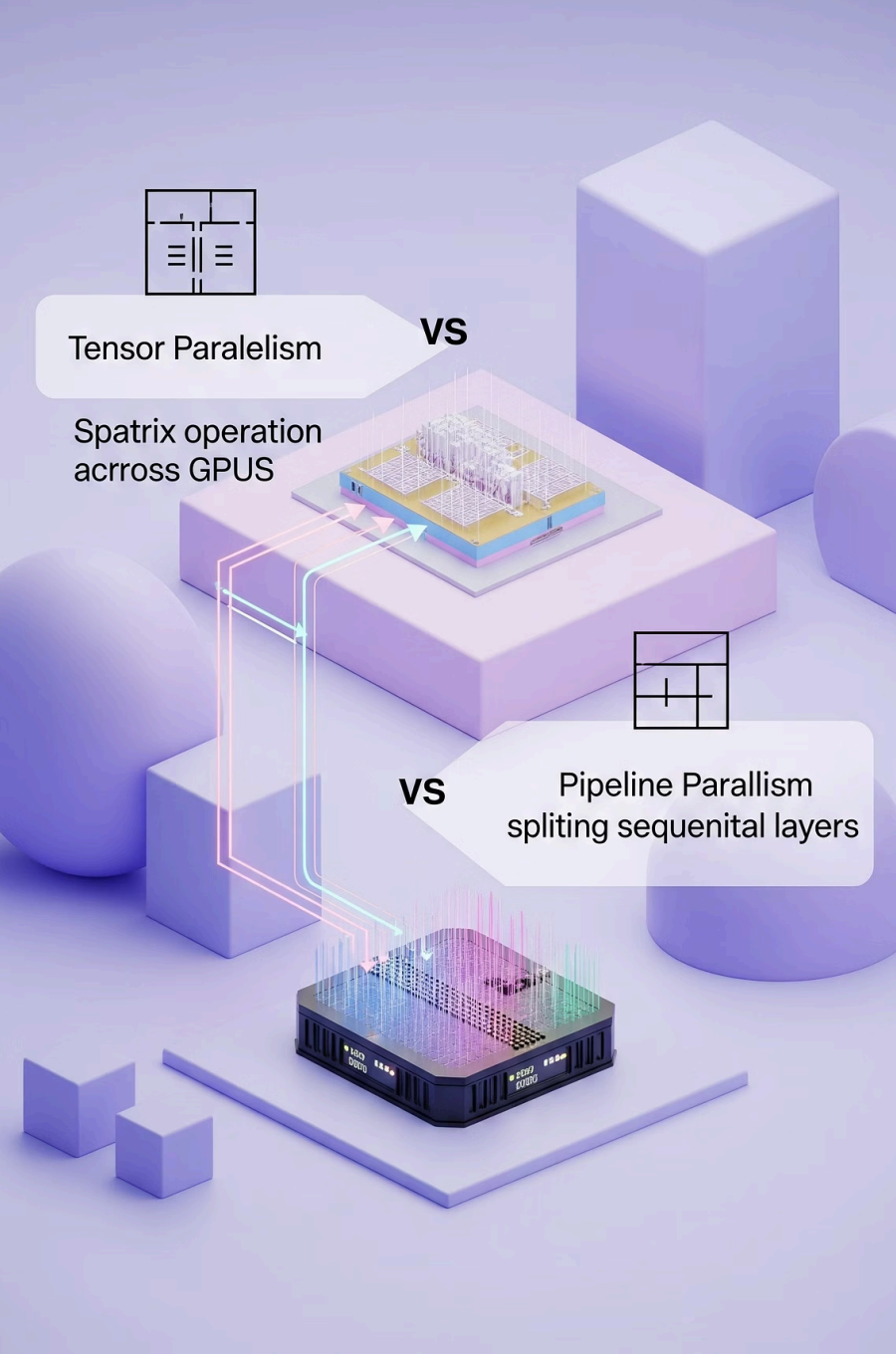
- Replicate the **entire model** across multiple GPUs
- Each GPU processes its own mini-batch of training data
- Gradients are averaged across all GPUs (all-reduce operation)
- Parameters updated synchronously after gradient averaging

Implementation Options:

PyTorch DDP, Horovod, DeepSpeed ZeRO-1



⊗ **Key Limitation:** Memory requirements don't scale - each GPU needs enough memory to hold the full model



Model Parallelism (MP)



Core Concept

Split model parameters across multiple GPUs, with each device holding only a portion of the network



Tensor Parallelism

Split individual tensors/matrices across devices (e.g., dividing attention heads)



Pipeline Parallelism

Split sequential layers across devices, with activations flowing through the pipeline

Model parallelism becomes **essential** when models are too large to fit in a single GPU's memory, but introduces challenges in balancing computation and minimizing communication.

Comparing Data vs Model Parallelism

Aspect	Data Parallelism (DP)	Model Parallelism (MP)
Memory Usage	Full model replica per GPU	Sharded model across GPUs
Scaling Limits	Limited by batch size	Can fit ultra-large models
Communication	Gradient synchronization (all-reduce)	Activation passing between GPUs
Implementation	Simple, robust, widely supported	Complex, requires topology awareness
Best For	Small/medium models (<10B parameters)	Large models (>20B parameters)

 Neither approach alone is optimal for frontier-scale models with 100B+ parameters

Hybrid Approaches: 3D Parallelism



Data Parallelism (DP)

Scale batch size across compute nodes



Tensor Parallelism (TP)

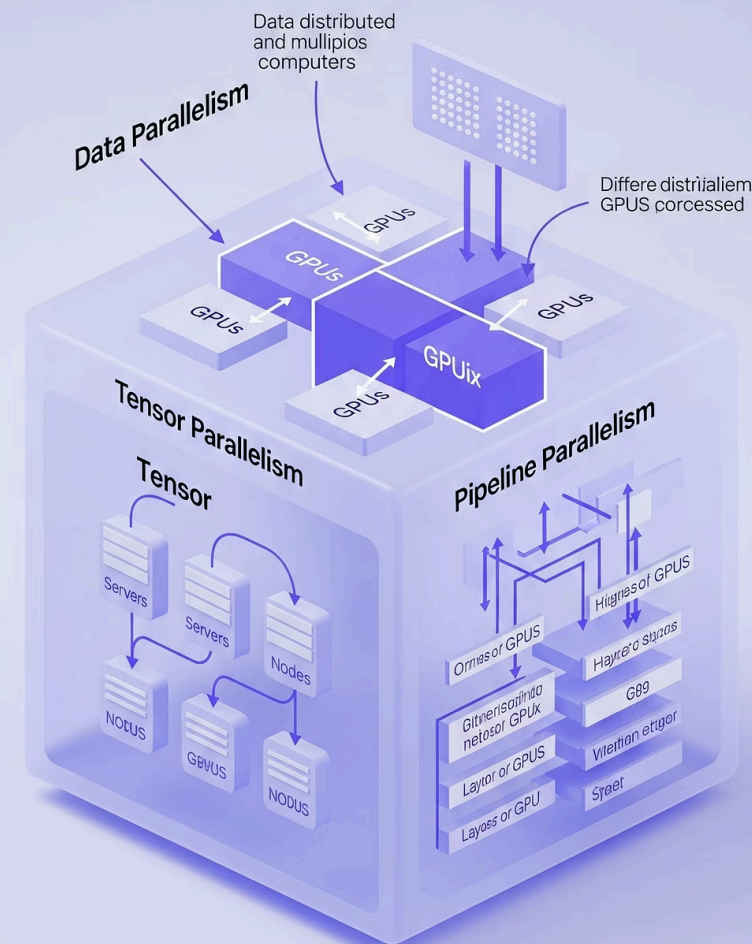
Shard large matrix multiplications within transformer blocks



Pipeline Parallelism (PP)

Distribute layers across pipeline stages to enable forward/backward bubbles

Modern frameworks like Megatron-LM, DeepSpeed, and Alpa combine these approaches to train models like [GPT-3 \(175B\)](#), [BLOOM \(176B\)](#), [MT-NLG \(530B\)](#), and [PaLM \(540B\)](#).



Memory Considerations

Memory Profile by Approach:

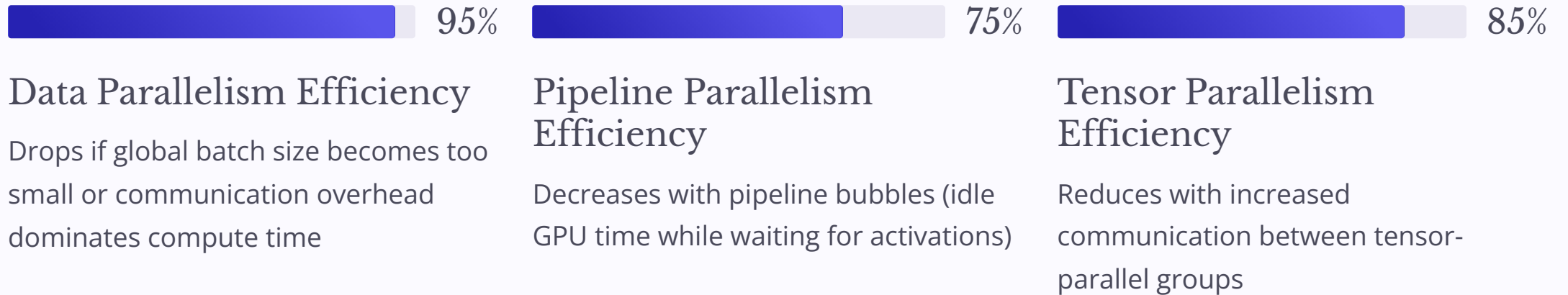
- **Data Parallelism:** High memory overhead (full parameters, gradients, and optimizer states on each GPU)
- **Model Parallelism:** Better memory scaling but increased communication overhead
- **Hybrid Approaches:** FSDP/ZeRO combine data parallelism with parameter sharding for better efficiency



⚠ For models exceeding 100B parameters, even sophisticated parallelism may need parameter offloading to CPU memory or NVMe storage during parts of training

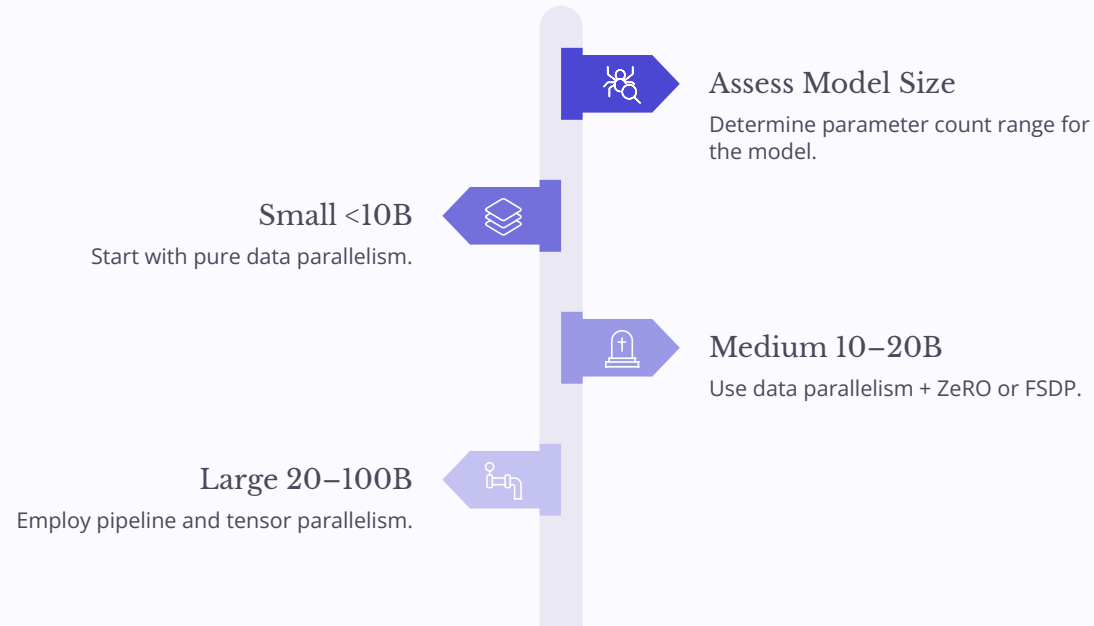
Different parallelism strategies significantly impact activation memory requirements, a critical factor for training stability and throughput.

Performance & Efficiency Tradeoffs



Optimal strategy depends on **GPU count**, **interconnect bandwidth**, and **model architecture**. Always measure **tokens/sec/GPU** as your real efficiency metric.

Best Practices



- Start simple with pure data parallelism for models under 10B parameters
- Introduce ZeRO/FSDP optimizations when memory becomes a constraint
- Add pipeline and tensor parallelism for models exceeding 20B parameters
- For 100B+ models, implement full 3D parallelism with careful topology mapping
- **Always benchmark before scaling up** - theoretical scaling rarely matches real-world performance

Key Takeaways

Data Parallelism

Easy to implement and scale out, but becomes memory-hungry for large models

First choice for most training scenarios under 10B parameters

Model Parallelism

Enables training of models that exceed single-GPU memory, but requires complex infrastructure

Critical for frontier models with specialized hardware configurations

Hybrid Approaches

All frontier-scale models (100B+) rely on sophisticated combinations of 3D parallelism

Represents the current state-of-the-art in scaling deep learning

Selecting the right scaling strategy requires balancing **memory utilization**, **communication patterns**, and **computational throughput** based on your specific model and infrastructure.

Next steps: Evaluate your current training workloads against these patterns to identify optimization opportunities.